SECTION 1

Eng\ mariyam khashapa

NUMPY LIBRARY

How can you optimize NumPy operations using broadcasting?
Answer : Broadcasting allows operations on arrays of different shapes without explicit loops.

```
import numpy as np
a = np.array([[1], [2], [3]])  # Shape (3,1)
b = np.array([4, 5, 6])  # Shape (1,3)
c = a + b  # Broadcasts 'b' to (3,3)

print(c)
# Output:
# [[5 6 7]
#  [6 7 8]
#  [7 8 9]]
```

What is the difference between np.dot() and np.matmul()?
Answer:

- np.dot(A, B): Performs dot product (for 1D arrays) or matrix multiplication (for 2D arrays).
- np.matmul(A, B): Always performs matrix multiplication, even for higher-dimensional arrays.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))     # Matrix multiplication
print(np.matmul(A, B))  # Same as above

x = np.array([1, 2])
y = np.array([3, 4])
print(np.dot(x, y))     # Dot product: 1*3 + 2*4 = 11
print(np.matmul(x, y))  # Error (matmul requires at least
one 2D array)
```

How can you generate structured arrays in NumPy for handling mixed data types?

Answer:
Structured arrays allow different data types in a single NumPy array.

```
dt = np.dtype([('name', 'U10'), ('age', 'i4'), ('weight', 'f4')])
data = np.array([('Alice', 25, 55.5), ('Bob', 30, 70.2)], dtype=dt)

print(data['name'])   # Output: ['Alice' 'Bob']
print(data['age'])    # Output: [25 30]
```

How does NumPy handle missing values efficiently?

Answer:

np.nan is used for missing values in floating-point arrays.

Use np.isnan() to check for NaN values.

Use np.nanmean(), np.nanstd(), etc., to ignore NaN values.

```
arr = np.array([1, 2, np.nan, 4, 5])
mean = np.nanmean(arr)  # Ignores NaN
print(mean)  # Output: 3.0
```

How do you perform in-place modifications in NumPy without copying data?

Answer:

Use operations with **out** parameter or np.copyto():

```
arr = np.array([1, 2, 3, 4])
np.add(arr, 10, out=arr)  # In-place modification
print(arr)  # Output: [11 12 13 14]
```

How can you find the eigenvalues and eigenvectors of a matrix?

Answer:

Use numpy.linalg.eig():

```
A = np.array([[4, -2], [1, 1]])
eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

How can you extract unique values and their counts in a NumPy array?

Answer:

Use np.unique() with return_counts=True:

```
arr = np.array([1, 2, 2, 3, 3, 3, 4])
unique_values, counts = np.unique(arr, return_counts=True)

print("Unique values:", unique_values)
print("Counts:", counts)
```

How can you generate a random matrix with a fixed sum?

```
A = np.random.rand(3, 3)
A = A / A.sum() * 100  # Adjust sum to 100
```

How do you efficiently filter elements using a boolean mask?

```
arr = np.array([10, 20, 30, 40, 50])
mask = arr > 25
filtered = arr[mask]

print(filtered)  # Output: [30 40 50]
```

NUMPY WITH DATASET

1. Loading a Dataset with NumPy

You can load a dataset (e.g., a CSV file) using numpy.loadtxt() or numpy.genfromtxt().

```
import numpy as np

# Load dataset (assuming numerical data)
data = np.loadtxt("data.csv", delimiter=",",
skiprows=1)  # Skip header row
print(data[:5])  # Print first 5 rows
```

2. Basic Statistics

NumPy provides several functions to analyze datasets.

```
# Example dataset
data = np.array([[10, 15, 20], [30, 35, 40], [50, 55, 60]])

# Compute mean, median, and standard deviation
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
```

3. Filtering Data

Extract specific data points based on conditions.

```
# Filtering values greater than 20
filtered_data = data[data > 20]
print(filtered_data)
```

4. Normalizing Data (Min-Max Scaling)

Normalization scales values between 0 and 1.

```
def min_max_normalize(data):
    return (data - np.min(data)) / (np.max(data) -
np.min(data))

normalized_data = min_max_normalize(data)
print(normalized_data)
```

5. Applying Functions to Data

NumPy's apply_along_axis() helps apply custom functions.

```
# Function to compute sum of each row
row_sums = np.apply_along_axis(np.sum, axis=1,
arr=data)
print(row_sums)
```

EXAMPLE

```
import numpy as np

# Step 1: Generate a Synthetic Dataset (10
rows, 5 columns)
np.random.seed(42)  # For reproducibility
data = np.random.randint(20, 60, size=(10,
5))  # Random values between 20 and 60
print("Dataset:\n", data)

# Step 2: Compute Basic Statistics
mean_values = np.mean(data, axis=0)
median_values = np.median(data, axis=0)
std_values = np.std(data, axis=0)

print("\nMean values:", mean_values)
print("Median values:", median_values)
print("Standard deviation:", std_values)
```

```python
# Step 3: Filtering Data - Find employees with salary > 45 (Column Index 2)
high_salary = data[data[:, 2] > 45]
print("\nEmployees with high salary:\n", high_salary)

# Step 4: Normalize Data (Min-Max Scaling)
def min_max_normalize(data):
    return (data - np.min(data, axis=0)) / (np.max(data, axis=0) -
np.min(data, axis=0))

normalized_data = min_max_normalize(data)
print("\nNormalized Dataset:\n", normalized_data)

# Step 5: Reshape and Transpose Data
reshaped_data = data.reshape(5, 2, -1)  # Reshape into a new 5x2x-1 form
transposed_data = data.T  # Transpose the dataset

print("\nReshaped Data (5x2x-1):\n", reshaped_data)
print("\nTransposed Data:\n", transposed_data)a1
```

```python
# Step 6: Compute Correlation and Covariance
correlation_matrix = np.corrcoef(data, rowvar=False)
covariance_matrix = np.cov(data, rowvar=False)

print("\nCorrelation Matrix:\n", correlation_matrix)
print("\nCovariance Matrix:\n", covariance_matrix)

# Step 7: Apply Custom Function - Sum of each row
row_sums = np.apply_along_axis(np.sum, axis=1, arr=data)
print("\nTotal score per employee:", row_sums)
```

TASKS

1. How do you efficiently generate a diagonal matrix from a 1D array?
2. How do you find the indices of the top k largest elements in an array?
3. How do you efficiently normalize a NumPy array?
4. Write a Python script using NumPy to analyze a dataset of student grades.

Generate a random dataset of 10 students × 5 subjects (values between 0 and 100).

Compute the mean, median, and standard deviation for each subject.

Identify students who scored above 80 in at least 3 subjects.

Normalize the dataset using Min-Max scaling.

Reshape the dataset to (5, 2, -1).

Compute the correlation matrix for the subjects